

Ultrasonically-driven lipid-coated microbubble dynamics

César Villa^{1,*}, Antoine de Vinck², Alvaro Solans², André Cruz Faria², and Marceau Landry²

¹ Author; ² Group members

December 13, 2025

The radial oscillations of ultrasonically driven lipid-coated microbubbles were studied using ultra-high-speed imaging synchronized with acoustic measurements. The bubble radius was extracted from shadowgraphy recordings at 10 Mfps and compared with theoretical models. The classical Rayleigh–Plesset equation was found to overestimate the oscillation amplitude. Improved agreement was obtained using the Marmottant shell model with a surface elasticity $\chi = 0.3 \text{ N m}^{-1}$ and a surface viscosity $\kappa_s = 1.2 \times 10^{-8} \text{ kg s}^{-1}$. These values are consistent with typical parameters reported for lipid-coated microbubbles.

Microbubbles | Acoustic oscillations | Rayleigh–Plesset | Marmottant |

Microbubbles are widely used in biomedical applications, in particular as ultrasound contrast agents and as carriers for targeted drug delivery. Thanks to their lipid-coated shell, therapeutic molecules can be attached to or embedded within the microbubble interface and transported through the bloodstream. When exposed to an external ultrasound field, these microbubbles can oscillate or rupture, enabling the localized release of drugs at specific sites such as the inner walls of blood vessels. This approach allows for precise spatial control of drug delivery while reducing systemic exposure and side effects.

According to the laboratory instructions, the objective of this experiment is to investigate the radial oscillations of a single lipid-coated microbubble subjected to an ultrasonic excitation. The influence of the shell mechanical properties is studied by considering surface elasticity and surface viscosity values within prescribed ranges, namely $\chi \in [0.3, 2] \text{ N m}^{-1}$ and $\kappa_s \in [1.5, 12] \times 10^{-9} \text{ kg s}^{-1}$. The bubble dynamics are observed using ultra-high-speed microscopy and subsequently modeled using the classical Rayleigh–Plesset equation and the Marmottant shell model, allowing for a direct comparison between experimental measurements and theoretical predictions.

Theoretical Background

The radial dynamics of a spherical bubble in an incompressible liquid are described by the Rayleigh–Plesset equation

$$\rho \left(R\ddot{R} + \frac{3}{2}\dot{R}^2 \right) = P_g(R) - P_0 - P_{ac}(t) - \frac{2\sigma}{R} - 4\mu\frac{\dot{R}}{R}, \quad [1]$$

where $R(t)$ is the bubble radius, $\rho = 998 \text{ kg m}^{-3}$ is the liquid density, $\mu = 10^{-3} \text{ Pa s}$ the dynamic viscosity, $P_0 = 101325 \text{ Pa}$ the ambient pressure, and $P_{ac}(t)$ the applied acoustic pressure. The gas pressure is modeled using a polytropic law with exponent $\kappa = 1.07$.

For lipid-coated microbubbles, the interface cannot be considered clean. The Marmottant model introduces a radius-dependent surface tension accounting for shell buckling, elastic

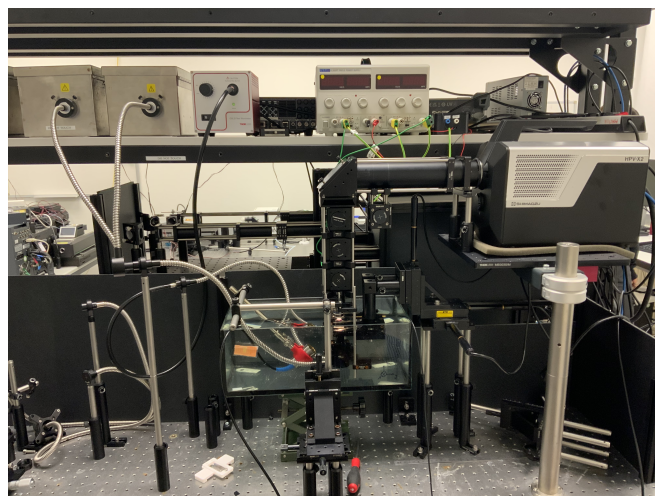


Fig. 1. Experimental setup used for ultrasound-driven microbubble experiments.

deformation, and rupture. The effective surface tension is zero in the buckled regime, increases linearly with surface area in the elastic regime with elasticity χ , and reaches the water surface tension $\sigma_w = 0.072 \text{ N m}^{-1}$ upon rupture. A surface viscosity κ_s is additionally introduced, providing extra damping. In this work, the initial surface tension is assumed to be zero due to shell stabilization.

Experimental Setup

The experiments were conducted in a water tank filled with water at room temperature. Lipid-coated microbubbles were excited using a focused ultrasound transducer (Precision Acoustics PA1612) operating at a centre frequency of 1.5 MHz with a focal distance of 85 mm. Assuming a sound speed of 1480 m s^{-1} in water, the corresponding acoustic time-of-flight to the focal region was approximately $57 \mu\text{s}$. This propagation delay was taken into account during data processing to ensure proper synchronization between acoustic excitation and optical measurements.

The transducer was driven with sinusoidal bursts of 20 cycles, corresponding to a waveform period of $T = 0.67 \mu\text{s}$ and a pulse duration of approximately $13 \mu\text{s}$. A reference calibration of the acoustic field was first performed at a peak-to-peak driving voltage of 100 mV under linear conditions. The acoustic pressure was measured using a needle hydrophone (Precision Acoustics 3686, sensitivity 39.1 mV/MPa), recorded

* cvilla@ethz.ch

with a digital oscilloscope, and converted into pressure units assuming a linear voltage-to-pressure relationship. Following calibration, three lipid-coated microbubbles with different equilibrium radii were analysed at a driving voltage of 200 mV, corresponding to an average focal pressure of approximately 60 kPa. In addition, one microbubble was investigated at a higher excitation amplitude of 600 mV to explore nonlinear oscillation regimes.

Bubble dynamics were recorded using an ultra-high-speed camera operating at 10 Mfps, providing a temporal resolution of 0.1 μs per frame. Each acquisition consisted of 256 frames, resulting in a total recording window of 25.6 μs , sufficient to capture the ultrasound burst and the subsequent bubble response. The camera was positioned perpendicular to the ultrasound propagation direction to minimize optical distortion. Shadowgraphy was used for visualization, with illumination provided by two optical fiber light guides triggered at $t = 49.5 \mu\text{s}$ and $t = 56 \mu\text{s}$, while the camera was triggered at $t = 54 \mu\text{s}$. This configuration extended the effective illumination duration and increased the number of usable frames during rapid bubble expansion and collapse.

Synchronization between the ultrasound excitation, illumination system, high-speed camera, and hydrophone acquisition was achieved using a common trigger signal. To align the pressure waveform with the recorded bubble dynamics, a fixed temporal delay of 54.8 μs was applied to the hydrophone signal during post-processing.

The bubble radius was extracted frame-by-frame using a custom image-processing algorithm implemented in MATLAB. Frames were converted to grayscale and normalized to compensate for illumination variations, and the bubble silhouette was segmented from the background. The projected area $A(t)$ was converted into an equivalent radius assuming axisymmetric dynamics as

$$R(t) = \sqrt{\frac{A(t)}{\pi}}.$$

A pixel-to-length calibration factor of 0.16 μm per pixel was used to convert image measurements into physical units. The resulting radius-time curves were used for comparison with theoretical predictions based on the Rayleigh-Plesset equation and the Marmottant shell model.

Results and Discussion

Figure 2 shows the results obtained for the first microbubble oscillation. The upper subplot displays the acoustic pressure signal driving the bubble, while the lower subplot compares the experimental radius-time evolution with the Rayleigh-Plesset and Marmottant models. The Marmottant model shows significantly better agreement with the experimental data, indicating that neglecting the shell elasticity χ and surface viscosity κ_s is not valid in this regime. The shell parameters χ and κ_s were obtained by minimizing the quadratic error between experimental and simulated radius-time curves. The optimization naturally converges toward a relatively low shell elasticity and a high surface viscosity, indicating that dissipative effects dominate over elastic restoring forces. A larger κ_s enhances damping and reduces the oscillation amplitude, while a smaller χ limits the elastic resistance of the shell during expansion, consistent with the attenuated oscillations observed experimentally. The pressure signal exhibits an acoustic pressure

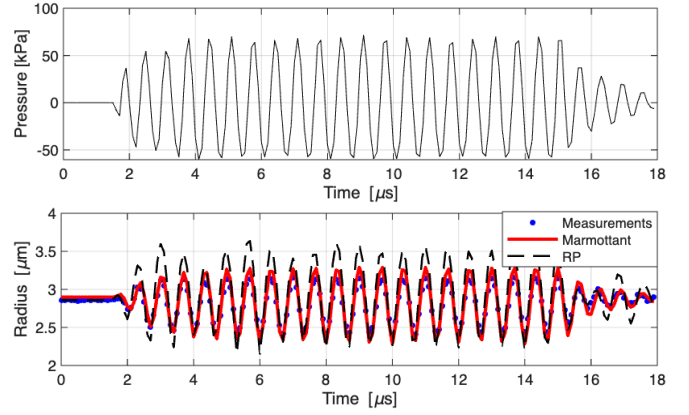


Fig. 2. (a) Acoustic pressure signal driving the bubble and (b) experimental radius-time evolution compared with Rayleigh-Plesset and Marmottant models.

Table 1. Equilibrium, maximum, minimum radii and maximum radial expansion measured for the different microbubbles.

Bubble	R_0 [μm]	R_{max} [μm]	R_{min} [μm]	ΔR_{max} [%]
1 (200 mV)	2.85	3.15	2.41	10.53
2 (200 mV)	3.63	3.79	3.36	4.41
3 (200 mV)	2.00	2.79	0.43	39.50
4 (600 mV)	1.77	2.83	0.38	59.89

amplitude of approximately 60 kPa, corresponding to instantaneous pressures approaching 2 bar when combined with the ambient pressure, thereby driving the bubble into a nonlinear oscillation regime.

The equilibrium, maximum, and minimum radii measured for the four investigated microbubbles are summarized in Table 1 with their maximal radial expansion. For the three bubbles excited at 200 mV, smaller equilibrium radii lead to larger relative oscillation amplitudes, highlighting the increasing influence of inertial effects, whereas larger bubbles exhibit more moderate oscillations due to enhanced damping and shell effects. This trend also suggests that smaller bubbles are driven closer to their resonance frequency, leading to larger relative oscillation amplitudes.

At a higher excitation amplitude of 600 mV, the corresponding high-speed video reveals very rapid expansion and collapse dynamics, consistent with the onset of strongly nonlinear oscillation regimes.

Conclusion

The radial oscillations of lipid-coated microbubbles driven by ultrasound were experimentally characterized using ultra-high-speed imaging and compared with theoretical models. While the Rayleigh-Plesset equation captures the oscillation frequency, it systematically overestimates the oscillation amplitude. Incorporating shell elasticity and surface viscosity through the Marmottant model provides a significantly improved agreement with the experimental data. The best agreement was obtained for a surface elasticity $\chi = 0.3 \text{ N m}^{-1}$ and a surface viscosity $\kappa_s = 1.2 \times 10^{-8} \text{ kg s}^{-1}$, consistent with values reported for lipid-coated microbubbles.

Appendix

Listing 1. MATLAB code: tracking + RP vs Marmottant comparison

```

1 % main_bubble.m
2 clear; clc; close all;
3
4 %% === Video selection ===
5 [videoFile, videoPath] = uigetfile({'*.avi;*.mp4','Video files
6     (*.avi;*.mp4)'}), ...
7     'Select a bubble video');
8 if isequal(videoFile,0)
9     disp('No video selected. ');
10    return;
11 end
12 videoName = fullfile(videoPath, videoFile);
13
14 %% === Video info ===
15 vidInfo = VideoReader(videoName);
16 nFramesTot = floor(vidInfo.Duration * vidInfo.FrameRate);
17 fprintf('Selected video: %s\n', videoName);
18 fprintf('Approx total frames (from file): %d\n', nFramesTot);
19 fprintf('File duration: %.3f s (%.2f fps)\n', vidInfo.Duration,
20     vidInfo.FrameRate);
21
22 %% === Frame window selection ===
23 prompt = { sprintf('Start frame (1..%d):', nFramesTot), ...
24     sprintf('End frame (1..%d, empty = end):',
25         nFramesTot) };
26 defaultans = {'1', num2str(nFramesTot)};
27 answer = inputdlg(prompt, 'Frame window', 1, defaultans);
28 if isempty(answer)
29     disp('No window defined. Abort. ');
30     return;
31 end
32 startFrame = str2double(answer{1});
33 endFrame = str2double(answer{2});
34
35 if isnan(startFrame) || startFrame < 1
36     startFrame = 1;
37 end
38 startFrame = round(startFrame);
39
40 if isnan(endFrame) || endFrame < startFrame || endFrame >
41     nFramesTot
42     endFrame = nFramesTot;
43 end
44 endFrame = round(endFrame);
45
46 fprintf('Tracking from frame %d to frame %d.\n', startFrame,
47     endFrame);
48
49 %% === Tracking on selected window ===
50 [~, R_px, center_px] = trackBubble(videoName, startFrame,
51     endFrame); % #ok<ASGLU>
52 nFrames = numel(R_px);
53
54 %% === Physical time: 1 frame = 0.1 us ===
55 dt = 0.1e-6;
56 t = (0:nFrames-1) * dt;
57 t_us = t * 1e6;
58
59 %% === Pixel to micrometer conversion ===
60 px_to_um = 0.16;
61 R_um = R_px * px_to_um;
62
63 %% === Keep only valid detections ===
64 valid = (R_px > 0);
65 t_us_valid = t_us(valid);
66 R_px_valid = R_px(valid);
67 R_um_valid = R_um(valid);
68
69 compare_RP_Marmottant(t_us_valid, R_um_valid);
70
71 %% === Display radius values ===
72 fprintf('\nMeasured radius frame by frame\n');
73 for k = 1:numel(t_us_valid)
74     fprintf('Frame %4d | t = %.3f us | R = %.2f px | R = %.3f
75         um\n', ...
76         k, t_us_valid(k), R_px_valid(k), R_um_valid(k));

```

```

73 end
74 fprintf('-----\n');
75
76 %% === Plot radius (pixels) ===
77 figure;
78 plot(t_us_valid, R_px_valid, 'o-', 'LineWidth', 1.2,
79     'MarkerSize', 4);
80 xlabel('Time [us]');
81 ylabel('Radius R(t) [pixels]');
82 title('Bubble radius evolution (pixels)');
83 grid on;
84
85 %% === Plot radius (micrometers) ===
86 figure;
87 plot(t_us_valid, R_um_valid, 'o-', 'LineWidth', 1.2,
88     'MarkerSize', 4);
89 xlabel('Time [us]');
90 ylabel('Radius R(t) [um]');
91 title('Bubble radius evolution (um)');
92 grid on;
93
94 % =====
95 % TRACKBUBBLE
96 % =====
97 function [t, R_t, center_t] = trackBubble(videoName,
98     startFrame, endFrame)
99
100     vid = VideoReader(videoName);
101     nFramesTot = floor(vid.Duration * vid.FrameRate);
102
103     if nargin < 2 || isempty(startFrame)
104         startFrame = 1;
105     end
106     if nargin < 3 || isempty(endFrame) || endFrame > nFramesTot
107         endFrame = nFramesTot;
108     end
109
110     startFrame = max(1, round(startFrame));
111     endFrame = max(startFrame, round(endFrame));
112
113     nFrames = endFrame - startFrame + 1;
114
115     t = ((startFrame-1):(endFrame-1)) / vid.FrameRate;
116     R_t = zeros(nFrames,1);
117     center_t = NaN(nFrames, 2);
118
119     % ROI selection at start frame
120     vid.CurrentTime = (startFrame-1) / vid.FrameRate;
121     IO = readFrame(vid);
122
123     figure('Name','ROI selection (double click to
124         validate)','Color','w');
125     imshow(IO, []);
126     title(sprintf('Select ROI around bubble (frame %d)',
127         startFrame));
128     hRect = imrect;
129     roiPos = wait(hRect);
130     roiPos = round(roiPos);
131     close;
132
133     figure('Name','Bubble tracking','Color','w');
134
135     % Frame 1
136     frameIdx = 1;
137     I = IO;
138     Iroi = imcrop(I, roiPos);
139     [centerLocal, R] = detectBubbleInROI(Iroi);
140
141     if R > 0 && all(~isnan(centerLocal))
142         center = [centerLocal(1) + roiPos(1) - 1, ...
143             centerLocal(2) + roiPos(2) - 1];
144     else
145         center = [NaN NaN];
146         R = 0;
147     end
148
149     R_t(frameIdx) = R;
150     center_t(frameIdx,:) = center;
151
152     imshow(I, []); hold on;
153     if R > 0
154         viscircles(center, R, 'EdgeColor','r','LineWidth',1);

```

```

151     plot(center(1), center(2), 'g+',
152           'MarkerSize',8,'LineWidth',1.5);
153 end
154 title(sprintf('Frame %d/%d (video frame %d) | R = %.2f px',
155             ...
156             frameIdx, nFrames, startFrame + frameIdx - 1,
157             R));
158 hold off; drawnow;
159 % Next frames
160 for frameIdx = 2:nFrames
161     if ~hasFrame(vid)
162         break;
163     end
164     I = readFrame(vid);
165     Iroi = imcrop(I, roiPos);
166
167     [centerLocal, R] = detectBubbleInROI(Iroi);
168
169     if R > 0 && all(~isnan(centerLocal))
170         center = [centerLocal(1) + roiPos(1) - 1, ...
171                 centerLocal(2) + roiPos(2) - 1];
172     else
173         center = [NaN NaN];
174         R = 0;
175     end
176
177     R_t(frameIdx) = R;
178     center_t(frameIdx,:) = center;
179
180     imshow(I, []); hold on;
181     if R > 0
182         viscircles(center, R,
183                   'EdgeColor','r','LineWidth',1);
184         plot(center(1), center(2), 'g+',
185              'MarkerSize',8,'LineWidth',1.5);
186     end
187     title(sprintf('Frame %d/%d (video frame %d) | R = %.2f
188                 px', ...
189                 frameIdx, nFrames, startFrame + frameIdx
190                 - 1, R));
191     hold off; drawnow;
192 end
193
194 % Post processing: center continuity filter
195 tolerance_speed = 5;
196
197 for i = 3:nFrames
198     if R_t(i-1) > 0 && R_t(i-2) > 0
199         dx_ref = center_t(i-1,1) - center_t(i-2,1);
200         dy_ref = center_t(i-1,2) - center_t(i-2,2);
201         d_ref = sqrt(dx_ref^2 + dy_ref^2);
202
203         if R_t(i) > 0
204             dx_cur = center_t(i,1) - center_t(i-1,1);
205             dy_cur = center_t(i,2) - center_t(i-1,2);
206             d_cur = sqrt(dx_cur^2 + dy_cur^2);
207
208             if abs(d_cur - d_ref) > tolerance_speed
209                 R_t(i) = R_t(i-1);
210                 center_t(i,:) = center_t(i-1,:);
211             end
212         elseif R_t(i) == 0 && R_t(i-1) > 0
213             R_t(i) = R_t(i-1);
214             center_t(i,:) = center_t(i-1,:);
215         end
216     end
217 end
218
219 % =====
220 % DETECTION IN ROI (illumination robust)
221 % =====
222 function [center, R] = detectBubbleInROI(Iroi)
223
224     center = [NaN NaN];
225     R = 0;
226
227     if size(Iroi,3) == 3
228         Igray = rgb2gray(Iroi);
229     else
230         Igray = Iroi;

```

```

228     end
229     I = im2double(Igray);
230     I = medfilt2(I, [3 3]);
231
232     radius_bg = 20;
233     se_bg = strel('disk', radius_bg);
234     I_bg = imclose(I, se_bg);
235     I_bg(I_bg == 0) = eps;
236
237     I_norm = I ./ I_bg;
238
239     bw = I_norm < 0.85;
240
241     bw = bwareaopen(bw, 50);
242     bw = imfill(bw, 'holes');
243     bw = imopen(bw, strel('disk', 2));
244
245     stats = regionprops(bw, 'Area', 'Centroid', 'Eccentricity');
246     if isempty(stats), return; end
247
248     [~, idxMax] = max([stats.Area]);
249     A = stats(idxMax).Area;
250     ecc = stats(idxMax).Eccentricity;
251
252     if A < 20 || ecc > 0.95
253         return;
254     end
255
256     CC = bwconncomp(bw);
257     if idxMax > numel(CC.PixelIdxList), return; end
258     bwMain = false(size(bw));
259     bwMain(CC.PixelIdxList{idxMax}) = true;
260
261     B = bwboundaries(bwMain, 'noholes');
262     if isempty(B), return; end
263
264     boundary = B{1};
265     if size(boundary,1) < 10
266         R = sqrt(A/pi);
267         center = stats(idxMax).Centroid;
268         return;
269     end
270
271     y = boundary(:,1);
272     x = boundary(:,2);
273
274     [xc, yc, Rfit] = fitCircle(x, y);
275
276     if isnan(Rfit) || Rfit <= 0 || Rfit > 500
277         R = sqrt(A/pi);
278         center = stats(idxMax).Centroid;
279     else
280         R = Rfit;
281         center = [xc, yc];
282     end
283 end
284
285 % =====
286 % FIT CIRCLE
287 % =====
288 function [xc, yc, R] = fitCircle(x, y)
289     x = x(:); y = y(:);
290     A = [2*x, 2*y, ones(size(x))];
291     b = x.^2 + y.^2;
292
293     if size(A,1) < 3
294         xc=NaN; yc=NaN; R=NaN; return;
295     end
296
297     p = A \ b;
298     xc = p(1); yc = p(2); c = p(3);
299     R = sqrt(c + xc.^2 + yc.^2);
300 end
301
302 % =====
303 % COMPARE RP VS MARMOTTANT (with optimization)
304 % =====
305 function compare_RP_Marmottant(t_exp_us, R_exp_um)
306
307     % Load CSV
308     default_filename = 'C1--1.5mhz-100mV-00002.csv';
309     if isfile(default_filename)
310         filename = default_filename;

```



```

312 else
313     [file, path] = uigetfile('*.csv');
314     if isequal(file,0), return; end
315     filename = fullfile(path, file);
316 end
317
318 raw_data = readmatrix(filename, 'NumHeaderLines', 5);
319 t_scope = raw_data(:,4);
320 V_scope = raw_data(:,5);
321 ok = ~isnan(t_scope) & ~isnan(V_scope);
322 t_scope = t_scope(ok);
323 V_scope = V_scope(ok);
324
325 % Voltage to pressure
326 sensitivity = 39.1e-3;
327 scaling_factor = 2.0;
328 P_scope = (V_scope / sensitivity) * 1e6 * scaling_factor;
329
330 offset = mean(P_scope(1:min(500,end)));
331 P_scope = P_scope - offset;
332 P_scope(abs(P_scope) < 3000) = 0;
333
334 [t_unique, idx] = unique(t_scope);
335 P_unique = P_scope(idx);
336 F_press = griddedInterpolant(t_unique, P_unique, 'linear',
    'nearest');
337
338 t_shift_phys = 54.8e-6;
339
340 % Experimental data
341 t_fit = t_exp_us(:) * 1e-6;
342 R_exp = R_exp_um(:) * 1e-6;
343 R0_base = mean(R_exp(1:min(5,end)));
344
345 % Parameters
346 param.rho = 998;
347 param.mu = 0.001;
348 param.kappa_g = 1.07;
349 param.sigma_w = 0.072;
350 param.sigma0 = 0.0;
351 param.p0 = 101325;
352 param.F_press = F_press;
353
354 % Optimization bounds
355 Ks_min = 1.5e-9; Ks_max = 12e-9;
356 Chi_min = 0.3; Chi_max = 2.0;
357 R0_min = 0.9; R0_max = 1.1;
358 DT_min = -5; DT_max = 5;
359
360 beta0 = [6e-9, 1.0, 1.0, 0.0];
361
362 odeOpts = odeset('RelTol',1e-4,'AbsTol',1e-7);
363
364 cost = @(b) objective(b, param, t_fit, R_exp, t_shift_phys,
    R0_base, ...
365                     Ks_min, Ks_max, Chi_min, Chi_max, ...
366                     R0_min, R0_max, DT_min, DT_max,
367                     odeOpts);
368
369 beta = fminsearch(cost, beta0);
370
371 Ks = min(max(beta(1),Ks_min),Ks_max);
372 Chi = min(max(beta(2),Chi_min),Chi_max);
373 R0 = R0_base * min(max(beta(3),R0_min),R0_max);
374 t_shift = t_shift_phys +
375     min(max(beta(4),DT_min),DT_max)*1e-6;
376
377 % Marmottant
378 pM = param;
379 pM.Ks = Ks;
380 pM.chi = Chi;
381 pM.R0 = R0;
382 pM.pg0 = pM.p0;
383 pM.t_shift_init = t_shift;
384
385 y0 = [pM.R0; 0];
386 [~, yM] = ode15s(@(t,y) RPM_rhs(t,y,pM), t_fit, y0);
387 R_M = yM(:,1);
388
389 % Rayleigh-Plesset
390 pR = param;
391 pR.R0 = R0_base;
392 pR.pg0 = pR.p0 + 2*pR.sigma_w/pR.R0;
393 pR.t_shift_init = t_shift_phys;

```

```

392
393 y0 = [pR.R0; 0];
394 [~, yR] = ode15s(@(t,y) RPM_rhs_Standard(t,y,pR), t_fit,
    y0);
395 R_RP = yR(:,1);
396
397 [R_M,~] = xcorr_align(t_fit, R_exp, R_M);
398 [R_RP,~] = xcorr_align(t_fit, R_exp, R_RP);
399
400 % Plots
401 figure('Color','w');
402
403 subplot(2,1,1)
404 plot(t_exp_us, R_exp_um,'b.','MarkerSize',10); hold on
405 plot(t_exp_us, R_M*1e6,'r','LineWidth',2)
406 plot(t_exp_us, R_RP*1e6,'k--','LineWidth',1.2)
407 xlabel('Time [us]'); ylabel('Radius [um]')
408 legend('Measurements','Marmottant','RP')
409 grid on
410
411 subplot(2,1,2)
412 P_plot = F_press(t_fit + t_shift);
413 plot(t_fit*1e6, P_plot/1000,'k')
414 xlabel('Time [us]'); ylabel('Pressure [kPa]')
415 grid on
416
417 fprintf('Optimized Chi = %.3f N/m\n', Chi);
418 fprintf('Optimized Ks = %.3e kg/s\n', Ks);
419 end
420
421 % =====
422 % COST FUNCTION
423 % =====
424 function J =
425     objective(b,p,t,Rexp,t0,R0,Ksmin,Ksmax,Chimin,Chimax,R0min,
    R0max,DTmin,DTmax,opts)
426
427 p.Ks = min(max(b(1),Ksmin),Ksmax);
428 p.chi = min(max(b(2),Chimin),Chimax);
429 p.R0 = R0 * min(max(b(3),R0min),R0max);
430 p.pg0 = p.p0;
431 p.t_shift_init = t0 + min(max(b(4),DTmin),DTmax)*1e-6;
432
433 try
434     y0 = [p.R0; 0];
435     [~,y] = ode15s(@(t,y) RPM_rhs(t,y,p), t, y0, opts);
436     J = mean((y(:,1) - Rexp).^2);
437 catch
438     J = 1e9;
439 end
440 end
441
442 % =====
443 % ALIGNMENT
444 % =====
445 function [Rout,dt] = xcorr_align(t,Rexp,R)
446 [cc,lags] = xcorr(Rexp-mean(Rexp),R-mean(R));
447 [~,i] = max(cc);
448 dt = lags(i)*mean(diff(t));
449 Rout = interp1(t+dt,R,t,'linear','extrap');
450 end
451
452 % =====
453 % RHS: MARMOTTANT
454 % =====
455 function dydt = RPM_rhs(t,y,p)
456 R = max(y(1),1e-9);
457 Rdot = y(2);
458
459 p_inf = p.p0 + p.F_press(t + p.t_shift_init);
460 pB = p.pg0*(p.R0/R)^(3*p.kappa_g);
461
462 R_buck = p.R0*sqrt(max(0, 1 - p.sigma0/p.chi));
463 if R <= R_buck
464     sigma = 0;
465 else
466     sigma = min(p.chi*((R/R_buck)^2 - 1), p.sigma_w);
467 end
468
469 visc_shell = 4*p.Ks*Rdot/R^2;
470 visc_liq = 4*p.mu*Rdot/R;

```

```

474     Num = pB - p_inf - 2*sigma/R - visc_liq - visc_shell -
475         1.5*p.rho*Rdot^2;
476     dydt = [Rdot; Num/(p.rho*R)];
477 end
478
479
480 % =====
481 % RHS: STANDARD RP
482 % =====
483 function dydt = RPM_rhs_Standard(t,y,p)
484     R = max(y(1),1e-9);
485     Rdot = y(2);
486
487     p_inf = p.p0 + p.F_press(t + p.t_shift_init);
488     pB = p.pg0*(p.R0/R)^(3*p.kappa_g);
489
490     visc = -4*p.mu*Rdot/R;
491     lap = -2*p.sigma_w/R;
492
493     Num = pB - p_inf + lap + visc - 1.5*p.rho*Rdot^2;
494     dydt = [Rdot; Num/(p.rho*R)];
495 end

```

1. P Marmottant, et al., A model for large amplitude oscillations of coated bubbles accounting for buckling and rupture. *The J. Acoust. Soc. Am.* **118**, 3499–3505 (2005).
2. A Prosperetti, A Lezzi, Bubble dynamics in a compressible liquid. part 1. first-order theory. *J. Fluid Mech.* **168**, 457–478 (1986).
3. E Stride, C Coussios, Cavitation and contrast: the use of bubbles in ultrasound imaging and therapy. *Proc. Inst. Mech. Eng. Part H* **224**, 171–191 (2010).